# API Testing with pytest and requests

This lab is individual homework. Work through it at home, then submit the link to your GitHub repository on MyCourses. Check the deadline there.

You will test the API of QuickPizza, the demo site from the K6 lab, with curl, Python, pytest and the requests library. Part 0 starts from a single request and adds one idea at a time, and every command comes with an explanation of what it does. Part A asks you to design test inputs with input space partitioning, as in Project 1, and to test who may change a rating. Part B asks you to explain what you found. Section 5 shows how each step carries over to the API testing part of Project 2.

## 1. Learning outcomes

This lab trains six skills.

1. Send requests to an API with curl and with the requests library, and read the status code, the headers and the JSON body of each response.
2. Write pytest tests that check the status code and the important fields of a response.
3. Use fixtures to create the data that each test needs, so that every test is independent of the others.
4. Test the create, read, update and delete operations of a resource, including the requests that the API must refuse.
5. Design test inputs with input space partitioning and boundary values, and write them as parametrized tests.
6. Recognise a defect in an API response, and write a defect report with the request, the expected result and the actual result.

## 2. Before you start

| | |
|---|---|
| **Tools** | Python 3.10 or newer, Git, a GitHub account, curl, and Visual Studio Code or another editor with a terminal |
| **Test site** | The QuickPizza API at https://quickpizza.grafana.com, the same demo site as in the K6 lab |
| **Reference** | requests documentation, https://requests.readthedocs.io, and pytest documentation, https://docs.pytest.org |

### 2.1 Create your repository

Keep all your work for this lab in one public GitHub repository named `api-lab`, so that the link you submit opens for the instructor.

1. Sign in at github.com and open github.com/new. Name the repository `api-lab`, choose Public, tick Add a README file, choose Python as the .gitignore template, and create it. The Python .gitignore stops Git from saving your virtual environment.
2. In Visual Studio Code, open the Command Palette, run Git: Clone, paste the HTTPS address of the repository, and choose a folder outside OneDrive, iCloud Drive and Google Drive.
3. Open the cloned folder and open a terminal in it with Terminal, New Terminal. Run every command in this sheet from that folder.

4. Commit and push after each step with the three commands below, changing the message each time.

```
git add .
git commit -m "Step 0.1"
git push
```

If Git asks for your name and email address at the first commit, set them with `git config --global user.name` and `git config --global user.email`.

### 2.2 Set up Python and curl

Create a file named `requirements.txt` in the repository with the three lines below. It lists the packages that the tests need, so that anyone can install them with one command.

```
pytest
requests
pytest-html
```

Create and activate a virtual environment, then install the packages. On macOS and Linux, type these lines.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

In Windows PowerShell, type these lines instead.

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

If PowerShell refuses to run the activation script, run `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser` once, then activate again. When VS Code asks whether to use the new environment for this workspace, choose Yes.

Check that curl is there with `curl --version`. Windows 10 and 11 include curl, but in Windows PowerShell the name `curl` belongs to a different command, so type `curl.exe --version` there, and `curl.exe` wherever this sheet says `curl`. This sheet was tested on 28 October 2026 with Python 3.13, pytest 9.1, requests 2.34, pytest-html 4.2 and curl 8.7.

### 2.3 Rules for QuickPizza

QuickPizza is shared by everyone who practises on it. Follow these rules.

- Create, change and delete only data that your own tests created. Never use the public token `abcdef0123456789`, or any token that you did not get by logging in, to create or change ratings, because those ratings belong to everyone. Project 2 has the same rule for the data of other groups.
- Use made-up user names and passwords, never real ones. The tests in this lab make names such as `lab-3f9c0a1b2d4e`.
- A full run of the tests in this lab sends about 140 requests and takes about a minute. Run it when you need to, but never in a loop.

## Part 0: API testing basics, one step at a time

An API is a set of addresses that a program calls instead of a person clicking on a web page. Each call is an HTTP request. A request names a method, such as GET to read something or POST to create something, and an address. It may also carry headers, which are extra lines such as a token, and a body, which holds data. The server sends back a response with a status code, headers and a body, which is usually JSON. Steps 0.1 to 0.4 send requests by hand with curl, Steps 0.5 to 0.8 send the same requests from Python and turn them into tests, and Steps 0.9 to 0.16 write tests that create their own data.

### Step 0.1: A first request with curl GUIDED

Type this command in the terminal and press Enter.

```
curl https://quickpizza.grafana.com/api/names
```

The address has two parts. `https://quickpizza.grafana.com` is the site, and `/api/names` is the path of one operation of its API. curl sends a GET request when you do not ask for another method, so this command asks QuickPizza to read its list of pizza names. The answer comes back on one line.

```
{"names":["Americana","Bolognese","Calzone", ... ,"Siciliana"]}
```

This is JSON. The braces hold an object, which is a set of named fields. Here the object has one field, `names`. The square brackets hold a list, and each item in the list is a piece of text in double quotes. Count the names, and write the number in RESULTS.md. Section 4 gives a template for that file.

### Step 0.2: The status code and the headers GUIDED

The answer in Step 0.1 showed only the body. Add the option `-i`, which tells curl to include the status line and the headers.

```
curl -i https://quickpizza.grafana.com/api/names
```

The output now has three parts. The first line is the status line, `HTTP/2 200`. `HTTP/2` is the version of the protocol, and `200` is the status code, which means that the request worked. Then come the headers, one per line, each with a name, a colon and a value. Find `content-type: application/json`, which says that the body is JSON, and find a line that starts with `set-cookie: AWSALB=`. Step 0.9 explains why this cookie matters. After an empty line comes the body, the same JSON as in Step 0.1.

The status code is the first thing that a test checks. The table lists the codes that you meet in this lab.

| Code | Meaning | Where you meet it |
|---|---|---|
| 200 OK | The request worked, and the body holds the answer | Reading the names, the tools, a pizza or a rating |
| 201 Created | The request created something new | A new user or a new rating |
| 204 No Content | The request worked, and the body is empty | Deleting a rating |
| 400 Bad Request | The input broke a rule of the API | A rating with 0 stars |
| 401 Unauthorized | The request has no valid token | A request without the Authorization header |
| 404 Not Found | The thing asked for does not exist | A rating that was deleted |

| Code | Meaning | Where you meet it |
|---|---|---|
| 500 Internal Server Error | The server failed while it handled the request | Step 0.16 |

### Step 0.3: A request that needs a token GUIDED

Most operations of QuickPizza need a token, which proves who is asking. Ask for the tools that QuickPizza uses to cut a pizza, without a token.

```
curl -i https://quickpizza.grafana.com/api/tools
```

The status line shows `401`, and the body is `{"error":"authentication failed"}`. QuickPizza refused the request because it carried no token.

A token travels in a header named Authorization. The token `abcdef0123456789` is the public one that QuickPizza gives for practice, as in the K6 lab. Create a file named `auth.txt` in VS Code that holds this one line.

```
Authorization: token abcdef0123456789
```

The line has the form of every header, a name, a colon and a value. Now send the request with the header.

```
curl -H "@auth.txt" https://quickpizza.grafana.com/api/tools
```

The option `-H` adds a header to the request. The `@` in front of the file name tells curl to read the header from the file, and the quotes stop PowerShell from treating the `@` as part of its own syntax. This time the answer is the list of tools.

```
{"tools":["Knife","Pizza cutter","Scissors"]}
```

Write in RESULTS.md the status code of the request without a token.

### Step 0.4: Sending data with POST GUIDED

A GET request only reads. To ask QuickPizza to make something, a program sends a POST request with data in its body. QuickPizza recommends a pizza when it receives the restrictions of a customer. Create `veg.json` with this one line, which asks for a vegetarian pizza.

```
{"mustBeVegetarian": true}
```

Send it with the option `-d`, which reads the body from the file and makes curl send a POST request instead of a GET.

```
curl -H "@auth.txt" -d "@veg.json" https://quickpizza.grafana.com/api/pizza
```

The answer describes one pizza on a single long line, with its name, its dough, its ingredients and the tool that cuts it. Every ingredient carries `"vegetarian":true`. Write the name of your pizza in RESULTS.md. You have now used the four parts of a request, the method, the address, a header and a body.

### Step 0.5: The same requests from Python GUIDED

Tests send requests from code, so try the same requests in the Python shell first. Check that the virtual environment from Section 2.2 is active, then start the shell.

```
python
```

The prompt changes to >>>. Type each line that starts with >>> yourself, without the >>>, and press Enter. A line without >>> shows what Python prints back. First load the requests library and keep the address of the site in a variable, so that you do not have to type it again.

```
>>> import requests
>>> site = "https://quickpizza.grafana.com"
>>> res = requests.get(site + "/api/names")
```

`requests.get` sends a GET request, the same request as in Step 0.1, and returns a response object. The variable `res` now holds that response. Ask it for the status code and for one header.

```
>>> res.status_code
200
>>> res.headers["Content-Type"]
'application/json'
```

Now read the body. `res.json()` turns the JSON text into Python values. A JSON object becomes a dictionary, and a JSON list becomes a Python list.

```
>>> data = res.json()
>>> type(data)
<class 'dict'>
>>> data["names"][0]
'Americana'
```

`data["names"]` takes the field `names` from the dictionary, and `[0]` takes the first item of that list. Next, send a request with a token. A header in requests is a dictionary that maps the name of the header to its value, and the parameter `headers` adds it to the request.

```
>>> auth = {"Authorization": "token abcdef0123456789"}
>>> res = requests.get(site + "/api/tools", headers=auth)
>>> res.status_code
200
>>> res.json()
{'tools': ['Knife', 'Pizza cutter', 'Scissors']}
```

Last, send the POST request from Step 0.4. The parameter `json` takes a Python dictionary, turns it into JSON text and sends it as the body, so you no longer need a file. Python writes `True` with a capital letter, and requests turns it into the JSON `true`.

```
>>> body = {"mustBeVegetarian": True}
>>> res = requests.post(site + "/api/pizza", json=body, headers=auth)
>>> res.status_code
200
>>> res.json()["pizza"]["name"]
'A Fine Fungi e Olive'
```

Your pizza will have a different name, because QuickPizza picks one at random. Leave the shell with `exit()`.

### Step 0.6: Your first test GUIDED

A test sends a request, as you did in the shell, and then checks the answer with `assert`. Create `test_names.py` with the code below.

```
import requests

SITE = "https://quickpizza.grafana.com"


# The list of pizza names answers 200 and holds Margherita.
def test_names_are_listed():
    res = requests.get(SITE + "/api/names")
    assert res.status_code == 200
    assert "Margherita" in res.json()["names"]
```

Read it line by line. `SITE` keeps the address of the site, as `site` did in the shell. The comment states the goal of the test, and Project 2 asks for such a comment above every test. The name of the function starts with `test_`, which tells pytest that it is a test. The test sends the request, then checks two things with `assert`. The first `assert` checks the status code. The second checks that the list of names holds Margherita. When the condition after `assert` is true, nothing happens. When it is false, the test fails. Run the test.

```
pytest -v
```

pytest looks for files whose names start with `test_`, runs every test function in them, and prints one line per test. The last line shows `1 passed`. Now see what a failure looks like. Change `"Margherita"` to `"Pineapple Supreme"`, and run `pytest -v` again.

The test now fails, and pytest shows the line of the `assert` that failed, marked with `>`, followed by an `AssertionError` that prints both sides of the comparison. The message tells you that `'Pineapple Supreme'` is not in the list of names. Reading this message is the first thing to do when a test fails. Change the name back to `"Margherita"`, and check that the test passes again.

### Step 0.7: A test with a token GUIDED

Create `test_tools.py`. It sends the request from Step 0.3 with the public token.

```
import requests

SITE = "https://quickpizza.grafana.com"
AUTH = {"Authorization": "token abcdef0123456789"}


# The list of tools answers 200 and names the three tools.
def test_tools_are_listed():
    res = requests.get(SITE + "/api/tools", headers=AUTH)
    assert res.status_code == 200
    assert res.json() == {"tools": ["Knife", "Pizza cutter", "Scissors"]}
```

`AUTH` holds the header, and `headers=AUTH` adds it to the request, as in the shell. The second `assert` compares the whole body with the dictionary that we expect, so one line checks every field and every tool. Run `pytest -v`. The summary shows `2 passed`, one test from each file.

### Step 0.8: Fields inside the answer, and a request that must be refused GUIDED

A test that checks the status code alone misses an API that answers fast but wrongly. Create `test_pizza.py`, which asks for a vegetarian pizza and looks inside the answer.

```
import requests

SITE = "https://quickpizza.grafana.com"
AUTH = {"Authorization": "token abcdef0123456789"}
VEGETARIAN = {"mustBeVegetarian": True}


# A vegetarian request gets a named pizza with vegetarian ingredients only.
def test_vegetarian_pizza():
    res = requests.post(SITE + "/api/pizza", json=VEGETARIAN, headers=AUTH)
    assert res.status_code == 200
    body = res.json()
    assert body["pizza"]["name"] != ""
    for ingredient in body["pizza"]["ingredients"]:
        assert ingredient["vegetarian"] is True, ingredient["name"]
```

`body = res.json()` keeps the dictionary, so the next lines can look inside it. The first check makes sure that the pizza has a name. The `for` loop then goes through every ingredient of the pizza and checks that it is vegetarian. The text after the comma, `ingredient["name"]`, is printed when the check fails, so a failing run names the ingredient that broke the rule. Run `pytest -v`. The summary shows `3 passed`.

A good API also refuses a request that it must not serve. Add this test to the end of `test_pizza.py`.

```
# A request without a token is refused with 401 and an error message.
def test_pizza_without_token_is_refused():
    res = requests.post(SITE + "/api/pizza", json=VEGETARIAN)
    assert res.status_code == 401
    assert res.json() == {"error": "authentication failed"}
```

It sends the same request without the token and expects status 401 with the error message from Step 0.3. Run `pytest -v`. The summary shows `4 passed`.

### Step 0.9: A user of your own, in the Python shell GUIDED

The tests from here on create ratings. Ratings made with the public token belong to everyone, so another student could change them while your test runs. Each test will therefore create its own user, log in, and use the token that it gets back. Try these steps in the Python shell first. Start it with `python`, and load two libraries.

```
>>> import requests, uuid
>>> site = "https://quickpizza.grafana.com"
>>> s = requests.Session()
```

A `requests.Session` sends requests like `requests.get` does, and it also keeps the cookies that the site sends back. QuickPizza runs on several servers behind a load balancer, and each server keeps its own users. The `AWSALB` cookie that you saw in Step 0.2 tells the load balancer which server to use, so a session that keeps it reaches the same server every time. Next, make a user name that nobody has used before.

```
>>> name = "lab-" + uuid.uuid4().hex[:12]
>>> name
'lab-5662445b4059'
```

`uuid.uuid4()` makes a random value, `.hex` turns it into letters and digits, and `[:12]` keeps the first 12 of them. Your name will differ. Create the user with a POST request.

```
>>> user = {"username": name, "password": "Pizza-2026"}
>>> res = s.post(site + "/api/users", json=user)
>>> res.status_code, res.json()
(201, {'id': 16002, 'username': 'lab-5662445b4059'})
```

Status 201 means that QuickPizza created the user. Now log in with the same name and password. The answer holds a token for this user.

```
>>> res = s.post(site + "/api/users/token/login", json=user)
>>> token = res.json()["token"]
>>> mine = {"Authorization": "token " + token}
```

`mine` is a header with your own token. Use it to read the ratings of the new user, then look at the cookies that the session keeps.

```
>>> res = s.get(site + "/api/ratings", headers=mine)
>>> res.status_code, res.json()
(200, {'ratings': []})
>>> s.cookies.keys()
['AWSALB', 'AWSALBCORS']
```

A new user has no ratings, so the list is empty. Leave the shell with `exit()`. The next two steps turn these lines into fixtures, so that every test can get a fresh user with one word.

### Step 0.10: One session for the whole run GUIDED

A fixture is a function that prepares something for a test. pytest runs the fixture before the test and hands its result to the test. Fixtures that many test files share go in a file named `conftest.py`, which pytest reads by itself. Create `conftest.py` with the code below.

```
import uuid

import pytest
import requests

SITE = "https://quickpizza.grafana.com"


class QuickPizza(requests.Session):
    """A session that puts the address of the site in front of each path."""

    def request(self, method, url, *args, **kwargs):
        return super().request(method, SITE + url, *args, **kwargs)


@pytest.fixture(scope="session")
def api():
    with QuickPizza() as session:
        yield session
```

Read it from the top. `uuid` is for the next step. `SITE` keeps the address of the site. Writing `SITE +` in front of every path gets long, so the class `QuickPizza` does it for us. It is a `requests.Session` with one change. Every request goes through the method `request` of the session, and this class puts

`SITE` in front of the path there and passes everything else on unchanged. A test can then write `api.get("/api/ratings")`.

`@pytest.fixture` marks the function `api` as a fixture. `scope="session"` asks pytest to build it once for the whole run, so every test shares one session and reaches the same server. `yield` hands the session to the tests, and the `with` block closes it when the run ends. Run `pytest -v`. No test uses the fixture so far, so the summary still shows `4 passed`.

### Step 0.11: A fixture that logs in a new user GUIDED

Add the code below to the end of `conftest.py`. It is the shell session of Step 0.9, written as a function.

```
def log_in_new_user(api):
    """Create a user with a new name, log in, and return its header."""
    name = "lab-" + uuid.uuid4().hex[:12]
    user = {"username": name, "password": "Pizza-2026"}
    res = api.post("/api/users", json=user)
    assert res.status_code == 201
    res = api.post("/api/users/token/login", json=user)
    assert res.status_code == 200
    return {"Authorization": "token " + res.json()["token"]}


@pytest.fixture
def auth(api):
    return log_in_new_user(api)
```

`log_in_new_user` makes a new name, creates the user, logs in and returns the header with the token, and its two `assert` lines stop at once if a step fails. The fixture `auth` calls it. A plain function is useful here because Step A4 needs a second user. A test asks for a fixture by naming it as a parameter. Create `test_users.py`.

```
# A new user can log in and starts with no ratings.
def test_new_user_has_no_ratings(api, auth):
    res = api.get("/api/ratings", headers=auth)
    assert res.status_code == 200
    assert res.json() == {"ratings": []}
```

The test names `api` and `auth`. pytest sees that `auth` needs `api` as well, builds the session first, then logs in a new user, and passes both to the test. Run `pytest -v`. The summary shows `5 passed`. Every test that names `auth` gets its own new user, so no test can disturb another.

### Step 0.12: A test without a session OPTIONAL

To see what the session protects you from, create `test_no_session.py`. It takes the same steps as `log_in_new_user`, but it sends each request on its own with `requests.post`, which keeps no cookies.

```
import uuid

import requests

SITE = "https://quickpizza.grafana.com"


# The steps of log_in_new_user, with every request on its own.
def test_log_in_without_a_session():
    name = "lab-" + uuid.uuid4().hex[:12]
    user = {"username": name, "password": "Pizza-2026"}
    res = requests.post(SITE + "/api/users", json=user)
    assert res.status_code == 201
    res = requests.post(SITE + "/api/users/token/login", json=user)
    assert res.status_code == 200
```

Run it three times with `pytest -v test_no_session.py`. Without the cookie, the load balancer may send the login to a server that has never seen the new user, and the login then fails with 401. In our tries on 27 and 28 October 2026, the login failed 46 times out of 49. A test that passes on some runs and fails on others, with no change to the code, is called flaky. Write your three results in RESULTS.md, then delete `test_no_session.py`, because a flaky test does not belong in your suite.

### Step 0.13: Create a rating GUIDED

A rating gives a pizza from 1 to 5 stars, so a test of ratings first needs a pizza. Add this fixture to the end of `conftest.py`.

```
@pytest.fixture
def pizza_id(api, auth):
    res = api.post("/api/pizza", json={}, headers=auth)
    assert res.status_code == 200
    return res.json()["pizza"]["id"]
```

`pizza_id` asks QuickPizza for a pizza, as in Step 0.4, and returns only its id. An empty dictionary means no restrictions. It names `auth`, so the pizza belongs to the server and the user of the test. Now create `test_ratings.py` with a first test.

```
# A new rating comes back with an id, its stars and its pizza.
def test_create_rating(api, auth, pizza_id):
    new = {"pizza_id": pizza_id, "stars": 5}
    res = api.post("/api/ratings", json=new, headers=auth)
    assert res.status_code == 201
    body = res.json()
    assert body["stars"] == 5
    assert body["pizza_id"] == pizza_id
    assert isinstance(body["id"], int)
```

The test builds the body of the new rating, sends it with POST to `/api/ratings`, and checks the answer. It expects status 201, the stars and the pizza that it sent, and an id, which QuickPizza gives every new rating. `isinstance(body["id"], int)` checks that the id is a whole number. Run `pytest -v`. The summary shows `6 passed`.

### Step 0.14: Read, change and delete a rating GUIDED

The other three operations need a rating that already exists. A fixture can make one before the test and remove it after the test. Add this fixture to the end of `conftest.py`.

```
@pytest.fixture
def rating(api, auth, pizza_id):
    """A rating with 3 stars, made before the test and deleted after it."""
    new = {"pizza_id": pizza_id, "stars": 3}
    res = api.post("/api/ratings", json=new, headers=auth)
    assert res.status_code == 201
    made = res.json()
    yield made
    api.delete(f"/api/ratings/{made['id']}", headers=auth)
```

The fixture creates a rating with 3 stars. `yield made` hands the rating to the test and waits. When the test ends, whether it passed or failed, pytest runs the line after `yield`, which deletes the rating, so no test leaves data behind. Now add three tests to the end of `test_ratings.py`.

```
# A saved rating can be read back by its id.
def test_read_rating(api, auth, rating):
    res = api.get(f"/api/ratings/{rating['id']}", headers=auth)
    assert res.status_code == 200
    assert res.json() == rating


# Changing the stars of a rating returns the new number of stars.
def test_update_rating(api, auth, rating):
    path = f"/api/ratings/{rating['id']}"
    res = api.put(path, json={"stars": 4}, headers=auth)
    assert res.status_code == 200
    assert res.json()["stars"] == 4


# A deleted rating can no longer be read.
def test_delete_rating(api, auth, rating):
    path = f"/api/ratings/{rating['id']}"
    res = api.delete(path, headers=auth)
    assert res.status_code == 204
    res = api.get(path, headers=auth)
    assert res.status_code == 404
```

The address of one rating ends with its id, for example `/api/ratings/4350`. The `f` in front of the text makes it an f-string, so `{rating['id']}` is replaced by the id. GET reads the rating and expects the same fields back. PUT changes the stars and expects the new number. DELETE removes the rating and expects 204, and the GET after it expects 404, because the rating is gone. Run `pytest -v`. The summary shows `9 passed`.

Each test gets a fresh user and its own rating from the fixtures, so it needs nothing from another test. Check this by running one test alone.

```
pytest -v test_ratings.py::test_delete_rating
```

It passes alone, as it does in the full run. A test that passes only after another test has run depends on the order of the tests, and Project 2 does not accept such tests.

### Step 0.15: More requests that must be refused YOUR TURN

Create `test_refused.py` with the three tests below. Each test creates the user that it needs with a new name, as in Step 0.9, and checks both the status code and the error message.

1. A second user with a name that is already taken gets status 400 and `{"error": "username already taken"}`.
2. Logging in with a wrong password gets status 401 and `{"error": "authentication failed"}`.
3. A user with an empty name gets status 400 and `{"error": "username field is empty"}`.

Run `pytest -v`. The summary shows `12 passed`.

### Step 0.16: Bug hunt checkpoint YOUR TURN

A customer can exclude tools, for example when there is no knife at home. The test below checks that the pizza never needs an excluded tool. `@pytest.mark.parametrize` runs the same test once for each input in its list, here once with `["Knife"]` and once with `["Scissors"]`. Create `test_excluded_tools.py`.

```
import pytest


# The pizza never needs a tool that the customer excluded.
@pytest.mark.parametrize("excluded", [["Knife"], ["Scissors"]])
def test_excluded_tool_is_not_used(api, auth, excluded):
    body = {"excludedTools": excluded}
    res = api.post("/api/pizza", json=body, headers=auth)
    assert res.status_code == 200
    assert res.json()["pizza"]["tool"] not in excluded
```

1. Run it. Both inputs pass, and the summary shows `14 passed`.
2. Both inputs exclude exactly one tool, so they test the same block. Take the characteristic "how many of the three tools are excluded" and write its blocks in RESULTS.md. For each block that the test misses, add one input to the list in `parametrize`, and run the test again.
3. One block makes QuickPizza answer 500 with an empty body. Send that request with curl to confirm it, with the JSON body in a file, as with `veg.json`.
4. The product owner says that a request that no pizza can satisfy gets status 400 and a JSON error message, never 500. Move the failing input out of `parametrize` into a test of its own that expects exactly this. The new test fails, and the failure shows the defect.
5. Mark that test with `@pytest.mark.xfail(reason="Defect 1: ...", strict=True)` and complete the reason. pytest then reports it as `xfailed`, an expected failure, so the rest of the suite stays green while the defect stays in view. With `strict=True`, the run fails as soon as QuickPizza fixes the defect, which tells you to remove the mark.
6. Write a defect report in RESULTS.md with a title, the request, the expected result, the actual result and how severe you think it is. Project 2 asks for the request, the expected result and the actual result of every defect that you find.

When you finish, `pytest -v` shows `16 passed, 1 xfailed`.

## 3. The ideas you are about to practice

### 3.1 API tests are integration tests

A unit test checks one function, with its neighbours replaced by stubs. An API test sends real requests to the running system, so the web server, the code behind each request and the stored data all take part. It checks that these parts work together through the same interface that the web pages and the mobile apps of the system use, which makes it a form of integration testing (Pezzè and Young, Software Testing and Analysis, Chapter 21). API tests skip the browser, so they run faster than tests through the web pages, but they cannot see what a user sees on the screen.

### 3.2 Input space partitioning for an API

The fields of a request are the inputs of an operation. Input space partitioning splits the possible values of each input into blocks, using characteristics taken from the rules of the API. The blocks of one characteristic must not overlap, and together they must cover every value (Ammann and Offutt, Introduction to Software Testing, Chapter 6 of the second edition and Chapter 4 of the first).

Base choice coverage picks one base block for each characteristic, usually the most common valid value. The base test uses every base block. Each further test changes one characteristic to one of its other blocks and keeps the rest at the base. With characteristics of 3, 4, 2 and 3 blocks, base choice needs 1 + 2 + 3 + 1 + 2 = 9 tests, while all combinations would need 3 × 4 × 2 × 3 = 72.

### 3.3 Boundary values

Faults gather at the edges of a rule. When a rule accepts 1 to 5 stars, test 0, 1, 5 and 6, the values on each side of both edges.

## Part A: Input space partitioning for ratings

### Step A1: An input domain model GUIDED

POST `/api/ratings` takes a body with two fields, `pizza_id` and `stars`, and a token in the header. QuickPizza accepts a whole number of stars from 1 to 5, a pizza that exists, and the token of a logged-in user. The table gives one input domain model for this operation, with four characteristics.

| Characteristic | Blocks | Values in the tests |
|---|---|---|
| C1 Value of `stars` | below 1, 1 to 5 (base), above 5 | 0, 3, 6 |
| C2 Type of `stars` | whole number (base), decimal, text, missing | 3, 3.5, "3", no field |
| C3 Pizza | exists (base), does not exist | a new pizza, 999999999 |
| C4 Token | the user's own (base), none, wrong length | from `auth`, no header, `token short` |

Copy the table into README.md. Project 2 asks you to explain in README.md how you derived your inputs, and a table like this one is that explanation for one operation.

Create `test_create_rating.py`. It holds the base test and the tests for the two other blocks of C1. One test function with `parametrize` covers both blocks, because both expect the same answer.

```
import pytest

STARS_ERROR = "number of stars must be between 1 and 5 (inclusive)"


# Base choice: 3 stars as a whole number, a real pizza, the user's token.
def test_base_choice_is_saved(api, auth, pizza_id):
    new = {"pizza_id": pizza_id, "stars": 3}
    res = api.post("/api/ratings", json=new, headers=auth)
    assert res.status_code == 201
    assert res.json()["stars"] == 3
    assert res.json()["pizza_id"] == pizza_id


# C1: a number of stars below 1 or above 5 is refused with a message.
@pytest.mark.parametrize("stars", [0, 6])
def test_stars_out_of_range(api, auth, pizza_id, stars):
    new = {"pizza_id": pizza_id, "stars": stars}
    res = api.post("/api/ratings", json=new, headers=auth)
    assert res.status_code == 400
    assert res.json() == {"error": STARS_ERROR}
```

Run `pytest -v`. The new file adds three passing tests. pytest names each case after its input, for example `test_stars_out_of_range[0]`, so a failure points to the block that broke.

### Step A2: Base choice coverage YOUR TURN

Complete `test_create_rating.py` so that it reaches base choice coverage of the model in Step A1. Add one test for each block of C2, C3 and C4 that is not the base, and keep the other characteristics at their base. Check the status code and the error message in each test, and find each message with curl first. When you are done, the file holds 9 tests, and `pytest -v` shows `25 passed, 1 xfailed`. In README.md, write next to each block the name of the test that covers it.

### Step A3: Boundary values for an update YOUR TURN

Create `test_update_rating.py` for PUT `/api/ratings/{id}`. Use the `rating` fixture, change its stars to 0, 1, 5 and 6, and check that 1 and 5 are saved with status 200 while 0 and 6 are refused with status 400. A refused update must leave the rating as it was, so read the rating back after each update and check its stars. Write it as one test with `parametrize` over pairs of stars and expected status. `pytest -v` then shows `29 passed, 1 xfailed`.

### Step A4: Who may change a rating YOUR TURN

A user must never read, change or delete the rating of another user. Add a second user to the end of `conftest.py`.

```
@pytest.fixture
def other_auth(api):
    return log_in_new_user(api)
```

Create `test_rating_owner.py` with three tests. In each, the owner makes a rating through the `rating` fixture, and the other user tries to read, change or delete it. Check that each attempt is refused with a status code from 400 to 499, then read the rating as its owner and check that nothing

changed. Write the exact status code of each attempt in RESULTS.md. `pytest -v` then shows `32 passed, 1 xfailed`.

### Part B: Reflection

Answer these questions in RESULTS.md.

1. Your tests talk to a real server over the internet. Name two kinds of failure that they can find and a unit test with stubs cannot. Then give one reason to write unit tests with stubs as well.
2. Why does every test create its own user with a new name, rather than use the public token? How will you apply the same idea to the shared shop or hotel in Project 2?
3. Explain in your own words why the tests send every request through one session. If you did Step 0.12, describe what happened in your three runs.
4. Which block of the excluded tools revealed the defect in Step 0.16? Why is a 500 with an empty body worse for an app that calls the API than a 400 with a message? Why does the test stay in the suite as `xfail` instead of being deleted?
5. How many tests does base choice coverage need for the model in Step A1, and how many would all combinations coverage need? Show how you counted.
6. In Step A4, which status codes did you get when the other user read, changed and deleted the rating? Which codes would you expect? Is the difference a defect? Explain.
7. Create `madeup.txt` with the line `Authorization: token 0000000000000000`, a made-up token of 16 characters, and run the command below. What comes back? Is this a defect, and what could a stranger do with it on a real system? Only read with this token, and never create or change anything with it.

   ```
   curl -H "@madeup.txt" https://quickpizza.grafana.com/api/ratings
   ```

8. Name one thing that an API test checks better than a test through the web pages, and one thing that only a test through the web pages can check.

## 4. Deliverables

Before the last push, save the output of a full run as an HTML report.

```
pytest -v --html=report.html --self-contained-html
```

Open `report.html` in a browser. It lists every test with its result and its duration, and it shows the details of each failure. Project 2 asks you to save the output of a full run in the same way.

Push everything to your `api-lab` repository, and submit its link on MyCourses before the deadline shown there. README.md explains how to install and run the tests, and it holds the input domain

model from Step A1 with the test that covers each block. The repository should look like this, plus the JSON files that you made for curl.

```
api-lab/
  .gitignore
  README.md
  RESULTS.md
  requirements.txt
  report.html
  auth.txt
  veg.json
  madeup.txt
  conftest.py
  test_names.py
  test_tools.py
  test_pizza.py
  test_users.py
  test_ratings.py
  test_refused.py
  test_excluded_tools.py
  test_create_rating.py
  test_update_rating.py
  test_rating_owner.py
```

Start RESULTS.md from this template.

```
# API lab results

## Part 0
- Step 0.1, the number of pizza names:
- Step 0.3, status code of the request without a token:
- Step 0.4, the name of your pizza:
- Step 0.12, results of the three runs without a session (optional):
- Step 0.16, blocks of "how many of the three tools are excluded":

## Defect report
- Title:
- Request:
- Expected result:
- Actual result:
- Severity:

## Part A
- Step A4, status codes when the other user reads, changes and deletes:

## Part B
1.
2.
3.
4.
5.
6.
7.
8.
```

## 5. From this lab to Project 2

The API testing part of Project 2 asks for the same skills on the shop or the hotel. The table shows where this lab practised each rule of the project.

| Project 2 asks for | Where you practised it |
|---|---|
| At least three operations, each with its own test suite | One test file per operation in Part A |
| Inputs from input space partitioning, explained in README.md | Steps A1 and A2 |
| A check of the status code and the important fields, and a comment that states the goal of each test | Every test in this lab |
| Independent tests that create the data they need | The fixtures in Steps 0.10 to 0.14 |
| One command that runs the whole suite | `pip install -r requirements.txt`, then `pytest -v` |
| The output of a full run, with an explanation of each failure | `report.html`, and the defect report in Step 0.16 |

The systems of Project 2 log in differently from QuickPizza. The API guides on MyCourses give the details for each system, and the four lines below show the parts of requests that they need.

```
res = requests.get(url, auth=(user, password))
token = res.headers["Cart-Token"]
res = requests.post(url, json=body, headers={"Cart-Token": token})
res = requests.get(url, params={"output_format": "JSON"})
```

`auth=` sends Basic auth, which the admin API of the shop and the API of the hotel use. `res.headers` reads a header of the answer, such as the `Cart-Token` that the Store API of the shop hands out, and `headers=` sends it back with the next request. `params=` adds query parameters to the address, such as the `output_format=JSON` that the hotel API needs to answer in JSON rather than XML. Groups that choose JUnit can write the same tests with the REST Assured library.

### 5.1 Keep passwords and keys out of your repository

The password or API key of your group must never appear in code that you push to GitHub, because anyone who reads the repository could then use it. Read it from an environment variable instead.

```
import os

USER = os.environ["SHOP_USER"]
PASSWORD = os.environ["SHOP_API_PASSWORD"]
```

Set the variables in the terminal before you run the tests. The names here are examples, so choose your own. On macOS and Linux, type the first two lines below, and in Windows PowerShell, type the last two. A new terminal forgets them, so set them again in each new terminal.

```
export SHOP_USER="your-group-user"
export SHOP_API_PASSWORD="paste-it-here"
$env:SHOP_USER = "your-group-user"
$env:SHOP_API_PASSWORD = "paste-it-here"
```